1. How do we mathematically calculate the best-fit parameters in linear regression? Which formula is used to directly compute linear regression weights?

The Core Concept

To find the best-fit parameters $\boldsymbol{\beta}$ in ordinary least squares (OLS) linear regression, we seek to find the parameter vector that minimizes the sum of squared errors (S) between the observed target values $\boldsymbol{y}$ and the predicted values $\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\beta}$.

Mathematical Derivation

The objective is to find $\hat{\boldsymbol{\beta}} = \operatorname{argmin}_{\boldsymbol{\beta}} S(\boldsymbol{\beta})$. We write the sum of squared residuals in matrix form:

$$S(\boldsymbol{\beta}) = (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}) = \boldsymbol{y}^T\boldsymbol{y} - \boldsymbol{y}^T\boldsymbol{X}\boldsymbol{\beta} - \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{y} + \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}$$

Since $\boldsymbol{y}^T\boldsymbol{X}\boldsymbol{\beta}$ is a scalar, it equals its transpose $\boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{y}$. Thus, the expression simplifies to:

$$S(\boldsymbol{\beta}) = \boldsymbol{y}^T\boldsymbol{y} - 2\boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{y} + \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}$$

To minimize $S(\boldsymbol{\beta})$, we take the partial derivative with respect to the parameter vector $\boldsymbol{\beta}$ (noting explicitly that the derivative is taken with respect to the parameters, not the data matrices) and set it to zero:

$$0 = \frac{\partial S(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = -2\boldsymbol{X}^T\boldsymbol{y} + 2\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}$$

Rearranging the terms yields the **Normal Equations**:

$$\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta} = \boldsymbol{X}^T\boldsymbol{y}$$

Direct Weight Computation Formula

By isolating $\hat{\boldsymbol{\beta}}$, we get the closed-form formula used to directly compute linear regression weights:

$$\hat{\boldsymbol{\beta}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$$

> **Note on Computational Complexity:** While algebraically elegant and straightforward to implement via libraries like `numpy.linalg.lstsq`, computing this direct solution requires inverting the matrix $(\boldsymbol{X}^T\boldsymbol{X})$. This carries an expensive computational complexity of $\mathcal{O}(n^3)$, making it highly inefficient for datasets with large numbers of features (n).

2. What is Stochastic Gradient Descent? Why is Stochastic Gradient Descent faster for large datasets? How do we update the weights in gradient descent?

Definition

Stochastic Gradient Descent (SGD) is an iterative optimization algorithm used to minimize a cost function $S(\boldsymbol{\beta})$. Unlike batch gradient descent, which computes the true gradient using the *entire* dataset simultaneously, SGD estimates the gradient and updates parameters incrementally, **sample by sample** (or using small mini-batches).

Weight Update Mechanism

In standard Gradient Descent, parameters move opposite to the direction of the cost function's gradient vector. The parameter vector is modified at each step according to the rule:

$$\boldsymbol{\beta}' \rightarrow \boldsymbol{\beta} - \eta \nabla S(\boldsymbol{\beta})$$

Where:

- $\boldsymbol{\beta}$ represents the current weights/parameters.
- η is the learning rate (a multiplicative scalar determining step size).
- $\nabla S(\boldsymbol{\beta})$ is the gradient vector of the loss function.

Why SGD is Faster for Large Datasets

- **Reduced Computational Cost per Step:** Batch gradient descent requires calculating gradients over millions of samples before performing a single parameter update. SGD updates weights after viewing only a single sample (or mini-batch), making the cost per iteration completely independent of the dataset size.
- **Faster Convergence:** Because it updates parameters immediately, it can find its way near an optimal minimum much quicker in early iterations, dodging data redundancy.
- **Memory Efficiency:** It prevents the system from running out of RAM because the complete dataset does not need to be loaded into memory to compute gradients.

3. What is a nonlinear least squares problem? Why can nonlinear optimization be difficult?

Definition

A **nonlinear least squares problem** is an optimization scenario where the model function $f(x, \boldsymbol{\beta})$ depends *nonlinearly* on its unknown parameters $\boldsymbol{\beta}$. The objective remains minimizing the sum of squared errors:

$$S(\boldsymbol{\beta}) = \sum_{i} \left(y_i - f(x_i, \boldsymbol{\beta})\right)^2$$

However, because f is non-linear with respect to $\boldsymbol{\beta}$, taking the derivative $\frac{\partial S}{\partial \boldsymbol{\beta}}$ does not produce a system of linear equations. It cannot be solved explicitly via a clean closed-form matrix inversion like $(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$.

Why Nonlinear Optimization is Difficult

1. **No Analytical Solution:** The equations must be solved via numerical, iterative approximations (e.g., Levenberg-Marquardt or Gradient Descent) rather than directly computed.
2. **Non-convex Optimization Landscapes:** The error surface often contains multiple local minima, saddle points, and plateaus. Algorithms can get trapped easily in a suboptimal local minimum instead of discovering the globally ideal parameters.
3. **High Sensitivity to Hyperparameters:** Iterative techniques heavily depend on finding a careful balance for parameters like learning rates; poor selections cause structural failure in parameter discovery.

4. Explain curve fitting using nonlinear least squares. Why is initialization important in nonlinear optimization?

Curve Fitting via Nonlinear Least Squares

Because nonlinear equations cannot be solved directly, we approximate them iteratively by linearizing the model function $f(x_i, \boldsymbol{\beta})$ around a neighborhood of a parameter guess $\bar{\boldsymbol{\beta}}$ using a first-order Taylor expansion:

$$f(x_i, \bar{\boldsymbol{\beta}} + \boldsymbol{\delta}) \approx f(x_i, \bar{\boldsymbol{\beta}}) + \boldsymbol{J_i}\boldsymbol{\delta}$$

Where $\boldsymbol{J_i} := \frac{\partial f(x_i, \boldsymbol{\beta})}{\partial \boldsymbol{\beta}}$ represents the rows of the **Jacobian matrix** (the matrix of partial derivatives of the model with respect to its parameters).

By substituting this linearization into the optimization objective and minimizing with respect to the step vector $\boldsymbol{\delta}$, we get a set of linear equations solved at each iteration to compute our parameter jump:

$$\left(\boldsymbol{J}^T\boldsymbol{J}\right)\boldsymbol{\delta} = \boldsymbol{J}^T\left[\boldsymbol{y} - \boldsymbol{f}(\bar{\boldsymbol{\beta}})\right]$$

The algorithm iteratively solves for $\boldsymbol{\delta}$, updates the parameters ($\bar{\boldsymbol{\beta}} \leftarrow \bar{\boldsymbol{\beta}} + \boldsymbol{\delta}$), and recomputes the Jacobian until convergence criteria are satisfied.

Importance of Initialization

Since these numeric optimization paths explore localized spaces, **initial parameter guesses are critical**:

- **Avoiding Local Minima:** An inappropriate initial guess will guide the optimizer down an unintended slope, trapping it permanently inside a bad local minimum.
- **Preventing Divergence:** If initial guesses place the algorithm within a region with flat gradients or extreme mathematical volatility, the parameter steps ($\boldsymbol{\delta}$) can become unstable, causing the model to diverge rather than converge.

5. What is learning rate η in gradient descent? What happens if the learning rate is too small or too large?

Definition

The **learning rate (η)** is a fundamental hyperparameter in gradient descent that scales the size of the step taken by the algorithm along the negative gradient during parameter updates. It dictates how aggressively or conservatively the model updates its weights.

If the Learning Rate is Too Small:

- The parameter adjustments are microscopic ($\boldsymbol{\eta} \nabla S(\boldsymbol{\beta}) \approx 0$).
- The optimization process progresses at an incredibly slow rate, costing vast computational time.
- The algorithm runs a significant risk of getting trapped permanently in the very first shallow local minimum or saddle point it encounters.

If the Learning Rate is Too Large:

- The step sizing overshoots the lower valley of the cost function.
- The path will aggressively bounce back and forth across the walls of the error surface, failing to converge.
- The loss value will structurally **diverge**, blowing up to infinity as updates become progressively chaotic.

6. What is a cost/loss function?

Definition

A **loss function** (often called a **cost function** when aggregated over an entire training set) is a mathematical metric that quantifies the error between a model's predictions ($\hat{y}$) and the true targeted benchmarks (y). It maps the performance of a machine learning model's specific parameter state down into a single real number.

Purpose in Machine Learning

The cost function acts as the foundational objective function for optimization algorithms. It creates an explicit error surface that algorithms traverse to minimize discrepancies, effectively defining what "best-fit" truly means for a specific task.

- **For Regression:** Common examples include Mean Squared Error (MSE) or Sum of Squares (S), which penalize squared distance errors.
- **For Classification:** Common examples include Binary Cross-Entropy / Log-Loss, which penalize predictive deviations from correct class probabilities.

7. What is overfitting/underfitting in machine learning? How can you identify an overfitted model? What causes overfitting? How can overfitting be reduced?

Definitions

- **Underfitting:** Occurs when a model is too structurally simple to grasp the underlying trends of the data. It performs poorly on both the training data and unseen testing configurations.
- **Overfitting:** Occurs when a model learns the training data *too well*, capture-fitting the random background noise, outliers, and statistical anomalies along with the core pattern.

Identifying Overfitting

An overfitted model is easily spotted by checking the divergence between training and testing errors:

- It yields an exceptionally low error (e.g., low MSE) on the **training set**.
- It exhibits a high error (e.g., high MSE) on the **validation or testing sets**.

Causes of Overfitting

- Using an excessively complex model relative to the simplicity of the underlying data pattern (e.g., high-degree polynomials or highly unconstrained Random Forests).
- A training dataset that is too small or lacks diverse representations.
- Training the optimization loop for too many epochs.

Methods to Reduce Overfitting

1. **Regularization:** Introduce L1 or L2 penalties (such as Ridge or Lasso regression) to limit parameter size explosions.
2. **Pruning:** Constrain tree architectures in algorithms like Decision Trees to prevent over-branching.

3. **Bootstrapping and Ensembles:** Combine predictions from multiple models via techniques like Random Forests.
4. **Gathering More Data / Model Validation:** Expand training examples or use systematic validation structures to correctly time early stopping points.

8. What is bias/variance in machine learning? Explain the bias–variance tradeoff. Which models typically have high bias? High variance?

Core Concepts

- **Bias:** The error introduced by approximating a complex real-world phenomenon with a model that is overly simplistic. High bias forces a model to miss relevant relations between features and outputs (**underfitting**).
- **Variance:** The amount by which a model's prediction function would shift if trained on a different underlying slice of data. High variance causes a model to model random noise instead of the real signal (**overfitting**).

The Bias-Variance Tradeoff

The total expected generalization error of a machine learning model can be broken down into:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

The **bias-variance tradeoff** represents a structural challenge: minimizing bias inherently amplifies variance, and vice versa.

- Increasing model complexity reduces bias but makes the model highly reactive to the training pool (raising variance).
- Simplifying the model insulates it from dataset fluctuations (lowering variance) but limits its flexibility to learn the true patterns (raising bias). The ideal configuration balances these to minimize total error.

Model Classifications

- **High Bias / Low Variance Models:** Simple parametric models, such as Linear Regression, Logistic Regression, and Linear SVMs.
- **High Variance / Low Bias Models:** Complex, flexible non-parametric models, such as deep Decision Trees , k-Nearest Neighbors with a small K value, and high-depth Random Forests.

9. What is bootstrapping in statistics and machine learning? Explain sampling with replacement. Why is bootstrapping

useful?

Definition

Bootstrapping is a statistical resampling technique that involves generating multiple simulated datasets by repeatedly drawing samples from an original master dataset.

Sampling with Replacement

Sampling **with replacement** means that when an individual sample is randomly selected from the original dataset to be placed into a new bootstrap group, it is not removed from the selection pool. It is placed right back into the original dataset, meaning a single data observation can be drawn multiple times within the same resampled dataset, while other observations may not appear at all.

Why Bootstrapping is Useful

1. **Quantifying Uncertainty:** It allows you to empirically measure error intervals and parameter uncertainties for models where analytical calculations are difficult (e.g., plotting confidence regions around a `KNeighborsRegressor` curve).
2. **Ensemble Power:** It forms the backbone of Bagging (Bootstrap Aggregation) algorithms like Random Forests, helping reduce overall model variance without damaging bias properties.
3. **Maximizing Small Datasets:** It provides a reliable way to simulate multiple data variations when collecting additional field data is too expensive or impossible.

10. How does logistic regression predict probabilities? Explain the decision boundary in logistic regression.

Probability Prediction

Though named a "regression" algorithm, Logistic Regression is fundamentally a linear classifier. Instead of mapping a linear combination of inputs directly to an unconstrained output, it pipes a linear score equation through the **Sigmoid (logistic) function**.

The linear model combination is defined as:

$$z = \boldsymbol{w}^T \boldsymbol{x} + b$$

The logistic sigmoid function transformations then compress z into a strict probability space bounded tightly between 0 and 1:

$$P(y = 1 \mid \boldsymbol{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

This output represents the estimated probability that a given data observation belongs to the positive class default.

The Decision Boundary

The **decision boundary** is the geometric line, plane, or hyperplane in feature space that separates different predicted classes.

Typically, a default classification threshold of 0.5 is applied:

- Predict Class 1 if $P(y = 1 \mid \boldsymbol{x}) \geq 0.5 \implies z \geq 0$
- Predict Class 0 if $P(y = 1 \mid \boldsymbol{x}) < 0.5 \implies z < 0$

Because the condition $z = \boldsymbol{w}^T \boldsymbol{x} + b = 0$ is a purely linear equation with respect to the input features $\boldsymbol{x}$, the decision boundary separating the classes forms a straight line or flat hyperplane. This explains why Logistic Regression is structurally classified as a **linear classifier**.